

With Great Powers: A Practical Guide to Agentic AI for Social Science Research

Simone Paci*

April 2026

Abstract

Research is undergoing a revolution. Agentic AI over the past year has drastically changed many social scientists' routines. Yet adoption has outpaced the building of shared best practices and standards, while uneven uptake threatens to introduce new sources of inequalities. This chapter offers a practical guide to catch up to speed as well as a template of effective and responsible use. It parses through the minimum vocabulary for agentic-AI literacy, builds a layered workflow that moves from tool choice through prompt, context, and harness engineering into agent management across the research process, and proposes two core governance principles. The first is *researcher control*, which aims at maintaining human ownership of every substantive decision; the second is *radical transparency*, which expands the replication package to include the inputs, throughputs, and outputs of the AI-assisted process. These principles are implementable today using the same infrastructure the paper recommends, and implementing them is the precondition for trust in agentic research workflows.

1 Introduction

In the space of a few months, a growing share of social scientists have restructured the day-to-day of their work around agentic AI. Recent surveys put researcher adoption of AI tools at 84% (Wiley, 2025); in parallel, AI-tagged publications in the natural, physical, and life sciences are growing at 26-28% year over year (Stanford Institute for Human-Centered Artificial Intelligence, 2026). The frontier is wider still. Agentic pipelines now run routine empirical work end to end, from data cleaning through heterogeneous-treatment sweeps to publication-ready tables (Pepinsky, 2026), and platforms built around agent orchestration are beginning to automate portions of the research process itself, from question formulation through analysis (Hall, 2026). Yet, uptake has outrun understanding. The problem is epistemic: when an agent drafts, runs, or interprets papers, the ordinary anchors of authorship and replicability no longer sit where long assumed by the research community.

Against a growing body of work that frames AI's implications for social science in programmatic or speculative terms, this paper takes the most concrete approach available: a working manual based on practical and feasible recommendations. I begin with the basic vocabulary a researcher needs to reason about the agentic landscape. In turn, I propose a set of best practices for effective AI deployment: a layered workflow that runs from tool choice through prompt, context, and harness engineering into agent-level management. Around this workflow, I advance an automated and thorough system of input, throughput, and output reporting that pushes the boundaries of current

*Stanford University. spaci@stanford.edu

replication standards. In its entirety, the approach is meant to be highly feasible and low-friction, helping level the playing field and begin already-long-overdue community-wide conversations about the future of research.

Running through both the workflow and the governance proposals are two principles that I argue should govern AI adoption in research: researcher control and radical transparency. The first holds that every substantive decision in a project, from theorizing to empirical design through interpretation and line-editing, must remain with the researcher. We may outsource execution, but not at the cost of ownership and judgment. The second holds that the burden of transparency and replication has shifted: releasing data and code is no longer sufficient, and a credible replication package must extend to the inputs, throughputs, and outputs of the AI-assisted process itself.

2 The AI Vocabulary

Using AI effectively starts with being able to define the tools being used. The rest of this section installs the minimum vocabulary the workflow in Section 3 will assume, organized into two layers: the tools themselves, and the engineering layers that require careful researcher intervention. Table 1 summarizes and exemplifies the key concepts.

2.1 Fundamentals

The obvious starting point is generative AI. Generative models produce text, code, and images from a prompt, and occupy a narrow slice of the wider AI field, which also houses predictive modeling, from basic OLS to classical machine learning. The practical shift in early 2026 consists of *agentic* generative AI: models wired into an environment that lets them call tools, read and write files, execute code, and loop until a task is done. Chat-based GenAI is limited: it produces text in response to user input¹. By contrast, agentic GenAI has, as the name suggests, agency: it can take actions, observe the outcome, and react to it iteratively. Keeping the underlying GenAI model equal, this shift unlocks a myriad of powerful applications.

A session with any such system is a *model-instance* with a bounded *context window*. The context window includes all input, throughput, and output ingested and produced by the model. Performance is not uniform across that window: as context fills, the model becomes less reliable, and long-running tasks therefore require either compaction (summarizing earlier turns into a condensed record) or a planned restart. Researchers who ignore this constraint discover it through drifted outputs and silent errors deep in a session. The immediate implication is that complex work is best split across multiple instances rather than crammed into one. One approach consists of agent *swarms*: a parent agent dispatches scoped tasks to child instances with fresh context and then integrates the results.

Much attention has been paid to model mistakes. I suggest structuring our thinking around two different typologies. “Hallucinations” can be thought of as false-positive errors: a citation that does not exist, a variable that was not in the dataset, a fact that was never in the source. These are conspicuous, but relatively easier to deal with: one needs “only” check the work. However, a second failure mode is also present and possibly harder to deal with: false-negative errors. This category includes all possible right research actions that were not taken by the agent: information never surfaced from a source document, a falsification check never included. This second mode is harder to catch and arguably more dangerous in research contexts: what is wrong is what is absent.

¹In practice, most chat interfaces now include agentic capabilities through a virtual sandbox, though usually not persistent and user-accessible

Again, the bad news is also good news: researcher still need to “do the work” and ensure that the agent has exhausted all avenues of correct research procedures.

Table 1: Key Concepts in the Agentic Generative AI Landscape

Concept	Definition	Example
<i>Fundamentals</i>		
Generative AI (GenAI)	Models that produce text, code, or images from a prompt. A narrow slice of the wider AI umbrella, which also houses predictive and classical ML.	Drafting a literature summary from a collection of PDFs.
Agentic AI	A GenAI model wired into an environment that lets it call tools, read and write files, execute code, and loop until a task is done.	Accessing a folder with a dataset, writing code to conduct a falsification check, and, if passed, running an analysis and producing a latex table.
Subagent / agent swarm	A parent agent spawns child instances with scoped tasks and fresh context, then integrates their outputs.	A research-lead agent dispatching parallel literature review subagents, one per research topic.
AI Failure Modes	Hallucinations or <i>false positives</i> : confident but fabricated output. <i>False negatives</i> : correct procedure never implemented.	Citing a paper that does not exist; Not considering an alternative interpretation of a result.
<i>Engineering layers</i>		
Prompt engineering	Crafting the instruction given to a model for a single task.	Specifying role, task, format, and worked examples for a text classification job.
Context engineering	Curating what the model can see: which files, prior turns, reference documents, and scratch space are visible at any moment.	A project folder with README, roadmap, background notes, and archive of prior research output.
Harness engineering	Designing the environment around the model: tools, skills, permissions, and control loops.	Configuring allowed shell commands and available skills.
Skill	A text file encoding the researcher’s preferred method for a recurring task.	A <code>lit-review-protocol</code> skill standardizing how every literature search is run and logged.
Orchestration	The researcher’s design for how tasks are split and routed across instances, subagents, and skills over a project.	Exploring measurement strategies for a given variable given a dataset: the lead agent manages the procedure while subagents perform analyses for each candidate variable.

2.2 Key Components

In the three years since the November 2022 ChatGPT moment, the best-practice frontier has moved in three stages that now provide the complex scaffolding around agentic AI deployment. The first stage, which coincided with the public arrival of ChatGPT, was *prompt engineering*: craft the

instruction well enough and the model will do the task. That skill remains necessary, but as context windows have grown the binding constraint has shifted outward. As models grew more powerful in 2024 and 2025, *context engineering* became the discriminant factor separating a useful session from a failed one. The context includes all information the model has access to in an instance: reference documents, prior output, relevant diagnostics. With the arrival of agentic AI, a third frontier has opened more recently around the runtime itself. *Harness engineering* is the design of the tools, skills, permissions, and control loops available to the model before any single prompt is written.

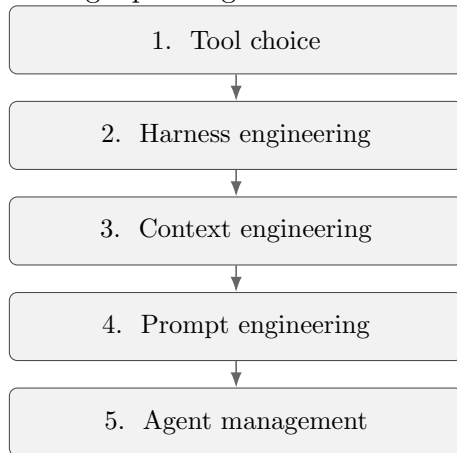
For practical purposes, two objects do most of the customization work inside the harness. A *skill* is a reusable, named procedure that an agent can invoke on demand: a short playbook encoding the researcher’s preferred method for a recurring task. *Orchestration* is the researcher’s overall design for how tasks are broken across instances, subagents, and skills over the course of a project. Together, they are the layer at which a generic model becomes a particular researcher’s collaborator, and where most of the practical leverage now sits.

As a useful metaphor, imagine an extremely capable research assistant. Given unclear instructions, blindfolded, and armed with nothing but pen and paper, that RA accomplishes little. Instead, equipped with a clear to-do list, a high-powered laptop, and a curated library of methods texts and prior work, the same person becomes a serious collaborator. Agents are no different. They need as precise a scaffolding as the researcher can provide – a task that the rest of this chapter focuses on.

3 How to Deploy Agentic AI Effectively

The setup stack is five layers deep, and each layer conditions the ones below it (see Figure 1). Tool choice determines which harnesses can exist on top; the harness determines which skills and tools the agent can reach; context engineering determines what that agent can see inside a given project; the prompt determines what the agent does with what it sees; and agent management determines how all of this gets reviewed and integrated across the research process.

Figure 1: Setting Up AI Agents in the Research Process



3.1 Choice of Toolset

The first researcher choice concerns which GenAI model and IDE (i. The current landscape offers several viable research harnesses, including Claude Code, Codex, Antigravity, and various IDE

plug-ins for VS Code and terminal-based environments.² Behind those harnesses sit at least three frontier providers in active competition, and the capability frontier has shifted several times in the last year alone. Committing a research workflow to a single provider is therefore fragile, and the more durable choice is to build practices that survive a model swap.

A second, less visible choice sits alongside the first. Closed-weights models remain at or near the frontier on most benchmarks, with open-weights models trailing by a modest margin ([Stanford Institute for Human-Centered Artificial Intelligence, 2026](#)). The gap narrows once replicability enters the picture. As Spirling and others have argued, research that leans on a commercial closed-weights model is effectively unreplicable once the vendor updates or retires that model; no amount of careful prompt logging recovers what the vendor has removed ([Spirling, 2023, 2026](#)). Open-weights models can be versioned, archived, and rerun. For any result meant to travel beyond proof-of-concept, that property is not a luxury.

3.2 Harness Engineering

The first set of choices about harness engineering concerns security. Agentic AI must be granted access to specific environments and permissions to perform specific actions. At one end of the spectrum, an agent could roam the entirety of a machine and perform any action a regular user could. This amount of freedom carries substantial risks: an agent could decide to delete years worth of work to make room for a large dataset needed to perform an analysis. On the other hand, the model could be quarantined to a sandboxed and given minimal permissions. The cost here becomes effectiveness and independence, with the researcher needing to approve every action. A larger discussion of security, beyond the scope of this chapter, involves questions of alignment and models’ increasing ability to circumvent restrictions ([Stanford Institute for Human-Centered Artificial Intelligence, 2026](#)).

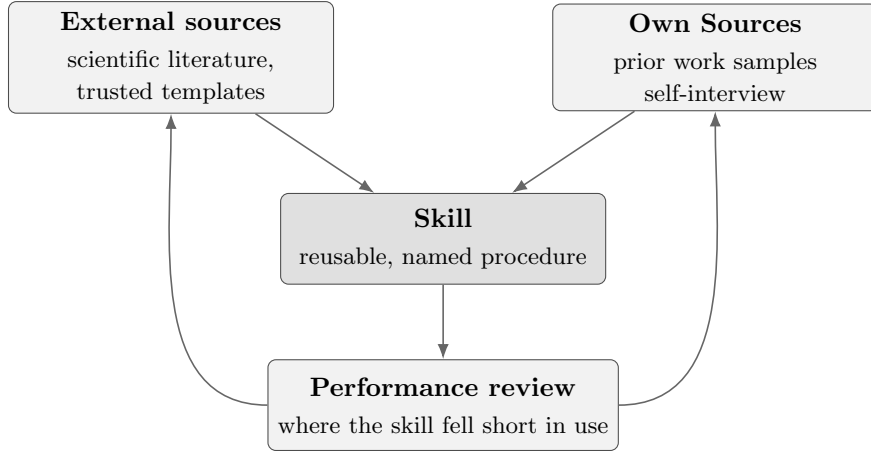
The central artifact inside a harness is the skill library. Through skills, researchers can routinely impose their preferred approach on agents’ behavior. Researchers should think of skills as transcending a specific task or project. Skills are reusable assets and should be centralized and managed across all AI-assisted projects. In practice, researchers should create a single, persistent repository of all their skills, so that agents can follow standard procedures for any given task. The goal of a personalized skill library should be understood as a long-term buildup. My practical advice is that each project should offer the opportunity to either create new skills or review old skills.

Skill-crafting itself should follow a set procedure that combines two types of inputs: external sources and researchers’ own knowledge. External sources include relevant literature, such as methods papers and authoritative template examples. Internal sources include samples of the researcher’s past work and a self-interviews to elicit the researcher’s idiosyncratic preferences and practical “tacit knowledge”. [Figure 2](#) illustrates this proposed approach, while the online appendix offers a template skill-building skill that researchers can use as a starting point for their own libraries.

Alongside clear access to a well-stocked skills library, a harness should include clear orchestration patterns. These are higher-level instructions on how to structure general workflows: complex processes into discrete steps, how to call the correct skills, how to engage the user throughout the process. The building of these documents, which are encoded in text files much like skills, can proceed following the skill-building system outlined above, but should rely on insights on the researcher’s work routines rather than their approach to any given specific task.

²These harnesses differ substantially in design, capability, and pricing, and most are proprietary. A feature-by-feature comparison is beyond the scope of this chapter.

Figure 2: A Procedural Approach to Skill-Building



3.3 Context Engineering

Once the harness is in place, the following setup step consists of what the agent should see. Context engineering treats the project as a sandbox repository: a single folder that carries all the relevant knowledge for the project, including background concepts, literature, data, scripts, and feedback. The user should also keep their work inside this folder, as if they were sharing it with an RA. Any new model instance that begins inside such a folder inherits the project’s full state, improving consistency, effectiveness, and unlocking transparency.

The first type of context engineering aims at making the context easy to navigate for agents through project-specific orchestration files. A well-structured README should state the purpose of the project, the structure of the folder, and any project-specific rules of engagement. A well-maintained implementation roadmap should function as a living to-do list and point of entry for any new model instance starting to work on the project – making spinning a new agent as simple as asking “what’s next?”. Finally, clear instructions on routine cleanups through a system of archive subfolders that absorb obsolete material rather than letting it accumulate. These keep the repository legible to any agent instance that enters it, including across long projects in which the researcher’s own memory has faded.

The second context-engineering goal deals with transparency. Every major asset in the folder should be recorded in an asset registry that flags it as **human**, **agent**, or **mixed**, along with a verification status ranging from **not-verified** through **partially-verified** to **human-verified**. Part of the project orchestration, implementable through the README file, should instruct agents to log all non-trivial session should be summarized in an interaction log that records date, researcher input, agent output, and model metadata. An emerging line of work on cryptographic authentication of research artifacts suggests these logs could eventually carry stronger guarantees (Gordon et al., 2025), but even the simpler CSV-based version already delivers a reconstructible record of the AI’s role.

Figure 3 renders its directory tree alongside the workflows layered on top. Such complex context scaffold may seem like a burdensome request of researchers. However, a *project-setup skill* can implement all of the above at startup, with minimal researcher intervention³. The same appendix also ships a structured literature-review protocol that extends the verification discipline to individ-

³In the online supplementary material, I provide two versions of this skill, one for new projects and one for projects already underway

Figure 3: Sample Project Context Setup

```
project/  
  README.md  
  implementation-roadmap.md  
  asset-registry.csv  
  interaction-log.csv  
  background/  
    literature/  
    concepts/  
    feedback/  
    archive/  
  drafts/  
  data/  
  figures/  
  tables/  
  scripts/  
  appendix/
```

Supporting workflows

- *Verification flags* on every major asset: **not-verified**, **partially-verified**, **human-verified**.
- *Provenance flags*: **human**, **agent**, or **mixed** for each asset entry.
- *Interaction log* summarizing every non-trivial agent session (date, input, output, model metadata).
- *Archive pattern*: obsolete files move to a local **archive/** rather than being deleted.
- *Project-setup skill*: scaffolds the structure above in a single invocation.

ual sources, governing how deep-research output is validated, deduped, and filed into the project’s bibliography and classification table. I wrote this chapter following this system and provide the full replication package as a template example.

I contend that this simple addition to a replication package would go a long way in establishing trust in an AI-supported research process. It provides a feasible way to track and report all inputs, throughput, and outputs related to a project. This expands our current understanding of a replication package beyond the simple verification of reproducibility of the quantitative analysis. Such expansion becomes crucial to validate that the locus of control and decision ownership remains with human researchers.

3.4 Prompt Engineering

Across specific tasks, the burden on prompt engineering is, at least in part, inversely proportional to quality of the harness and task-related skills. However, good prompting remains the bedrock of research *intent* and thus a necessary lever. Prompts, especially large, task-defining ones, provide the goals and focus that agents anchor their work on, within the constraints and guidelines set by the higher-level engineering steps. Managing sessions through prompts – that is “chatting with the agent” – is how the researcher stirs the ship and ensures a quality research output.

Instructions should be as precise as possible and might even be construed as the most substantive part of the researcher’s work in its interactions with agentic AI. Agents will fill whatever gaps in the goals or methods stated by the user, and the model’s decisions will be shaped by priors the researcher cannot easily audit. A prompt that does not specify how a task should be done is a prompt that delegates method to the model. A good plan is goal-oriented and articulated iteratively before the agent begins executing. Subplans for each major stage are usually worth the time they take to write, because they replace a long and drifting session with a sequence of short more controllable and accountable ones.

Concretely, researchers can refine their initial prompt through self-interview conducted collaboratively with the agent, akin to the skill-building process. The initial step could be as easy as stating “I need to do X, let’s design a detailed plan for this. Ask me questions about each aspect of X that I should provide instructions for.” My recommendation would be to start with more substantial instructions, as a purely agent-directed self-interview relies on the agent’s understanding

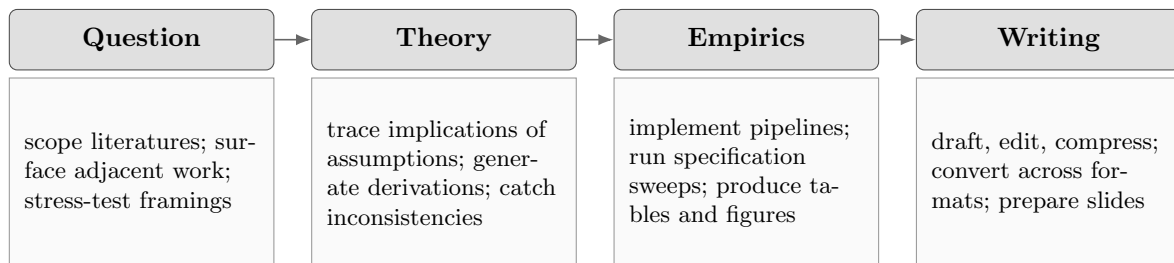
of what important components of the task are.

A final golden rule here is *one instance for one task*. Long-running sessions that mix unrelated work decrease agent effectiveness by accumulating context drift, cross-contaminating reasoning, and in turn make control harder. Researchers should always start a fresh session before switching to a new task.

3.5 Agent Management across the Research Process

Beyond the setup and best practices above, agentic AI integration requires a sequence of decisions made at every stage of the research process. The first, obvious question pertains when to deploy agents. Figure 4 sketches how all research stages may offer candidate applications for delegation to AI agents. As a general principle, grounded both in questions of effectiveness and control, the researcher should weight the balance between procedural and substantive nature of the task, understanding these aspects as non mutually exclusive and only partially inversely proportional to each other. Agents will be strong contenders for tasks that can be performed by following a simple checklist, say making the tikz package produce table 2. On the other hand, whenever decisions are substantive, say the structure and content of said figure, agents quickly become more unreliable options.

Figure 4: Research Stages and Candidate Agentic AI Applications



The second management aspect concerns monitoring and verification of AI output. I suggest three layers to this mandate. The first is dual chain-of-thought and final output review: current GenAI systems display the intermediate steps that the agent goes through as they work on a task. Parsing through the agent’s reasoning may flag mistakes and drift between the researcher’s intent and the agent’s final work. The second involves formulating as many secondary validation checks as possible, such as testing the internal coherence of the output. The third step is implementing as much of the above through a team of specialized review agents, equipped with skills developed for performing smoke tests of specific task categories.

4 Conclusions

This chapter has taken a deliberately workflow-level view of what AI means for social science, and much of my approach can be defended on the ground of accuracy and effectiveness, given current frontier model capabilities. However, in doing so, I have left aside the bigger questions: what becomes of the scientific enterprise when significant portions of its labor can be automated, what is the value of a human researcher is when a model can reliably automate much of the research process, and what the academic sector should reward when research output becomes cheaper to produce. These will not be answered in a manual. They will be answered by a messy evolving of disciplinary practice and the many conversations of the future months and years.

But the proposed workflow implies at least partial answers. At the most abstract level, I would argue that the two most fundamental aspects of the scientific enterprise are the direction of research and the error rate of findings. Human control of substantive decisions and radical transparency over the AI-assisted research process are meant to defend these principles.

The transparency principle is the most immediately actionable and possibly controversial recommendation. My stance is that the AI moment calls for an expanded notion of a replication package: not only the data and code that produced a final result, but the inputs, throughputs, and outputs of the AI-assisted process that shaped it. What the researcher prompted, which model and version answered, what the agent decided to do, and what the researcher changed afterward all belong in the record.

That is a substantial expansion of the reporting burden, both for authors and for reviewers who must engage it. The consoling point is that the same tools that raise the bar also help meet it. Interaction logs can be maintained in-session rather than reconstructed afterward; asset registries can be updated by the same agents that produce the assets; project-setup skills can scaffold the transparency infrastructure before research begins rather than bolting it on at submission. On the reviewer side, though beyond the scope of this chapter, the same logic could apply to standardizing and automating the systematic review of this process.

There are also further risks that remain unaddressed but bear mentioning. The first is to researchers themselves. Agents can do almost every operational task involved in producing a paper, and that possibility threatens a slow erosion of the underlying researcher’s skills. Second, agentic workflows compress research timelines in ways that may undermine the slow, deep thinking social-science training has long relied on, and this paper offers no principled way to calibrate the trade-off between speed and depth. Third, uneven adoption is likely to widen productivity gaps between researchers and institutions (Mohammadi et al., 2026; Chugunova et al., 2026), though the relatively low marginal cost of frontier subscriptions offers a partial democratizing countercurrent. Finally, environmental externalities, privacy exposures, and the broader societal consequences of automating knowledge work are real and important, though beyond the scope of this paper.

As a parting thought, what is prescribed here will age – perhaps very quickly. Much of the specific vocabulary, and some of the specific architectural advice, will likely be outdated within six months. The most fundamental meta-skill for researchers in this period is the capacity to keep up with a frontier that moves so quickly. Individual researchers cannot carry that burden alone, and they should not have to. Departments should treat AI-readiness as an institutional responsibility on both sides: in training, through coursework and apprenticeship that normalize workflows of the kind this paper describes. Similarly, the research community should develop a shared research infrastructure, through shared harnesses, maintained skills libraries, and review routines that make credible reporting feasible at scale.

References

- Chugunova, M., Harhoff, D., Hölzle, K., Kaschub, V., Malagimani, S., Morgalla, U., and Rose, R. (2026). Who uses AI in research, and for what? Large-scale survey evidence from germany. *Research Policy*, 55(2):105381.
- Gordon, S. C., Samii, C., and Su, Z. (2025). Data-NoMAD: A tool for boosting confidence in the integrity of social science survey data. arXiv preprint.
- Hall, A. (2026). The 100x Research Institution.
- Mohammadi, E., Thelwall, M., Cai, Y., Collier, T., Tahamtan, I., and Eftekhari, A. (2026). Is generative AI reshaping academic practices worldwide? A survey of adoption, benefits, and concerns. *Information Processing & Management*, 63(1):104350.
- Pepinsky, T. B. (2026). Agentic AI and social science research practice. Blog post.
- Spirling, A. (2023). Why open-source generative AI models are an ethical way forward for science. *Nature*.
- Spirling, A. (2026). 2025–2026 talks on replication, open models, and social-science uses of language models. Talks at Stanford Political Science and Yale ISPS.
- Stanford Institute for Human-Centered Artificial Intelligence (2026). Artificial intelligence index report 2026. Technical report, Stanford Institute for Human-Centered Artificial Intelligence.
- Wiley (2025). AI adoption jumps to 84% among researchers as expectations undergo significant reality check. Press release.

Online Appendix

The online appendix collects four complementary artifacts: a startup checklist for researchers opening a first agentic project (§A); structured descriptions of the three skill protocols that scaffolded this chapter’s production (§B); the schemas and sample entries of the three CSV ledgers that materialize the radical-transparency principle (§C); and an inventory of the full replication package (§D). The machine-readable sources for the skills and ledgers ship with the replication package.

A Startup Checklist

A one-page checklist for researchers already using chat-mode AI who are opening a first agentic project. The five thematic blocks map onto the setup stack of §3; the closing block runs on the cadence between projects rather than inside any one of them.

Tool.

- ☐ Install one harness (Claude Code, Codex, Antigravity, Cursor, or equivalent). Learn one before comparing.
- ☐ Authenticate and run a trivial read / write / execute test.

Harness.

- ☐ Copy one existing skill from your skill library. Do not start from scratch.
- ☐ Wire one deterministic validation check into the workflow: a type check, a reference count, a schema validation, a unit test.

Context.

- ☐ Scaffold a project folder with the `project-setup.md` protocol.
- ☐ Populate `background/` with your papers, data, and prior drafts.
- ☐ Initialize `asset-registry.csv` and `interaction-log.csv` and commit to using them from the first session.

Prompt.

- ☐ Start a fresh instance scoped to one task.
- ☐ Write the plan before the agent runs. The plan is the substantive work.
- ☐ Run the validation check at the end, not the agent's self-report.

Agent management.

- ☐ Inspect the agent's chain of thought, not only its final output.
- ☐ Sign off explicitly before committing any agent-produced artifact.
- ☐ Log the session in `interaction-log.csv` before closing the instance.

Before the next project.

- ☐ Note the skill gaps this project surfaced. Draft the missing skills with `skill-writing.md` before the next deadline.
- ☐ Archive obsolete files. Do not delete.

B Skill Protocols

The replication package ships three Markdown skills that this chapter's production exercised end to end: `project-setup.md`, `skill-writing.md`, and `lit-review-protocol.md`. Each is an interview-driven playbook rather than a prose essay. An agent reads the skill on invocation, walks the researcher through its phase-ordered interviews, and commits artifacts only after explicit sign-off. The three compose: `project-setup` scaffolds the folder the others write into; `skill-writing` produced the `lit-review-protocol` (and itself); and `lit-review-protocol` runs as a subagent inside `skill-writing` whenever a new skill needs external grounding.

B.1 `project-setup.md` — scaffold a transparent project folder

Invoked at project inception. The skill asks for the project title, type (empirical research, position paper, literature review, data exploration, or methodological), and any project-specific rules, then (i) creates the directory layout of Figure 3, pruning branches that do not apply; (ii) generates a filled-in `README.md` carrying the nine universal rules of engagement listed below; (iii) translates the researcher's outline into a checkable `implementation-roadmap.md`; and (iv) initializes the two

transparency ledgers `asset-registry.csv` and `interaction-log.csv`, whose schemas are given in §C. Together, these artifacts are the operational expression of the radical-transparency principle: every asset in the project has a row and every non-trivial session has a row, and both are created at inception rather than reconstructed at submission.

1. **Researcher retains control.** No substantive decision (design, interpretation, final phrasing, model or method choice) is made by an agent without explicit user sign-off.
2. **One instance, one task.** Start each non-trivial task in a fresh agent instance with a focused plan. Do not accumulate unrelated threads in a single context.
3. **Read-only source material.** Never modify files in `inputs/` (empirical projects) or `background/` source material. Derived artifacts go elsewhere.
4. **Register every new asset** (draft, figure, table, script, dataset, note, reference) in `asset-registry.csv` with a creator flag (`human` / `agent` / `mixed`) and verification status (`not-verified` / `partially-verified` / `human-verified`).
5. **Log every non-trivial session** in `interaction-log.csv` with date, input and output summaries, model metadata, and assets affected.
6. **Archive, don't delete.** Move obsolete files to the nearest `archive/` subfolder rather than removing them.
7. **Consult** `implementation-roadmap.md` before starting work and update it as steps complete.
8. **Skill orchestration.** For any complex task, first check the skill library. If no skill fits, ask the user whether to create one before improvising.
9. **Validation by default.** Prefer deterministic validation checks (tests, reproducibility runs, cross-source comparisons) over agent self-assessment.

B.2 `skill-writing.md` — write a new skill

Invoked when a recurring task keeps being re-specified, an existing skill has aged, or a team wants to institutionalize a shared practice. The protocol runs six phases: (0) identify the need and write a scope statement; (1) gather artifacts from the researcher's own prior work and dispatch a subagent to extract recurring moves, recurring decisions, and implicit rules; (2) gather external sources, calling the lit-review protocol as a subagent when outside grounding is load-bearing; (3) polish-the-skill interview, surfacing preferences, anti-patterns, and review cadence that the artifacts did not settle; (4) draft the skill following a standard skeleton (triggers, artifacts, universal rules, phases, handoffs, failure modes, worked example); and (5) test on one real task and revise. The central anti-pattern the skill warns against — and the one most often surfaced on first invocation — is the *training-memory skill*: a skill that reads as a textbook summary of the genre rather than as this researcher's practice, and that is the default output whenever the artifact-mining and self-interview phases are skipped.

B.3 `lit-review-protocol.md` — structured agent-assisted literature review

Invoked when a paper needs a new literature review, an existing review needs expansion, or raw deep-research output needs to be normalized and filed. Four active phases plus a maintenance

phase: (1) scope the strands — turn the paper’s outline into a named list of strands with rationale, target sections, anchor sources, and qualitative constraints, approved before any search runs; (2) draft prompts — one per strand, built by merging a permanent agent-facing template with a per-strand context block (research questions, anchors, scope overrides, exclusions); (3) ingest — validate BibTeX, resolve every DOI, dedupe against existing `references.bib`, append verified entries, and open a row in `classification.csv` tagged `partially-verified` until the researcher reads; (4) classify and map — every source gets `strand`, `sections`, `evidence_type`, `reliability`, and `read_status`, and sources slated for citation get a per-source Markdown note. Phase 5 maintains the review continuously as drafting surfaces new sources. Two hard rules together implement radical transparency on the literature side: never invent a DOI or venue, and never promote a source to `human-verified` until the researcher has read it.

C Ledger Schemas and Sample Entries

The three CSV ledgers that materialize the radical-transparency principle all ship with the replication package. This section reports their schemas (columns, admissible values, short gloss) and a handful of sample rows drawn directly from the files that produced this chapter. Long free-text fields are abridged for display.

C.1 asset-registry.csv

Column	Values	Gloss
asset_path	free text	Relative path from project root.
asset_type	draft figure table script dataset note reference other	Provenance flag. Model ID + harness, e.g., <code>claude-opus-4-7 / claude-code</code> . Blank if human.
creator	human agent mixed	
model_metadata	free text	
created	YYYY-MM-DD	Purpose, known issues, review status.
last_modified	YYYY-MM-DD	
verification	not-verified partially-verified human-verified	
notes	free text	

Table 2: Schema of `asset-registry.csv`.

asset_path	creator	verification	notes (abridged)
background/concepts/outline.tx	human	human-verified	Source-of-truth outline.
tables/table1-vocabulary.tex	agent	not-verified	Table 1 v2; ten rows in two blocks.
appendix/skill-writing.md	mixed	partially-verified	Full six-phase skill; expanded by agent from an 18-line human brainstorm.
background/literature/references.md	mixed	partially-verified	Seeded with four anchor entries; Zotero import target.
ai-agent-draft.tex	agent	not-verified	Snapshot of first full-paper drafting pass, preserved for diffing.

Table 3: Sample rows from the `asset-registry.csv` of this chapter’s project folder. The `asset_type`, `model_metadata`, `created`, and `last_modified` columns are omitted for space.

C.2 interaction-log.csv

Column	Values	Gloss
date	YYYY-MM-DD	
session_id	short identifier	e.g., 2026-04-21-a.
harness	claude-code chatgpt codex antigravity other	
model	model ID	e.g., claude-opus-4-7.
researcher_input_summary	1-2 sentences	
agent_output_summary	1-2 sentences	
assets_affected	comma-separated paths	Should match entries in asset-registry.csv.
notes	free text	

Table 4: Schema of `interaction-log.csv`.

session	model	researcher_input (abridged)	agent_output (abridged)
2026-04-21-d	claude-opus-4-7	Finalize the Phase-1 strand list: accept the five landscape strands plus transparency, control, cognitive effects, and ethics; split AI-methods into two strands; merge the trust-related strands.	Rewrote <code>lit-review-strategy.md</code> with the approved eight-strand list; reorganized <code>lit-review-prompts.md</code> ; reclassified four anchor sources in <code>classification.csv</code> .
2026-04-21-h	claude-opus-4-7	Enrich the commented outline, draft the chapter section by section, author the four figures and Table 1.	Wrote the full v1 of the chapter (abstract, §1-§4, appendix stub), authored four TikZ figures inline, and copied <code>main.tex</code> to the project root as <code>ai-agent-draft.tex</code> .

Table 5: Sample rows from the `interaction-log.csv` of this chapter’s project folder. The `date`, `harness`, `assets_affected`, and `notes` columns are omitted for space; input and output summaries are further abridged.

C.3 classification.csv

Column	Values	Gloss
citekey	free text	Matches <code>references.bib</code> .
strand	comma-separated strand IDs	From <code>lit-review-strategy.md</code> .
reliability	high medium low	Flagged by the search agent per the prompt template.
sections	comma-separated	Outline section numbers, e.g., 1.1, 3.1.
evidence_type	empirical conceptual methodological normative review	
read_status	unread skimmed read	
verification	not-verified partially-verified human-verified	
note_file	path	Per-source Markdown note, if one exists.
notes	free text	One-line hook.

Table 6: Schema of `classification.csv`.

citekey	strand	reliability	sections	evidence_type	read_status
gordon2025datanomad	A2	high	3.3, 4.2	methodological	skimmed
hai2026index	L1, A2, B2	high	1.1, 3.1, 4.1, 4.2	empirical	skimmed
pepinsky2026agentic	A1, A3	low	1.1, 3.5, 4.1	normative	read
spiraling2023nature	A2	high	3.1, 4.2	normative	skimmed

Table 7: Sample rows from the `classification.csv` of this chapter’s project folder, showing the four anchor sources seeded at project inception. All four currently sit at **partially-verified**; the Stage 6 review pass promotes cited sources to **human-verified**. Strand IDs refer to the eight-strand scheme in `lit-review-strategy.md`.

D Replication Package

The full replication package is the project folder that produced this chapter. It ships as a Dropbox mirror at `AI in Social Science Research/` and is available from the author on request. Beyond the three skills described in §B and the ledgers catalogued in §C, the package contains:

- `README.md` and `implementation-roadmap.md` — the filled-in rules of engagement and the living to-do list agents consulted throughout the project.
- `asset-registry.csv` — one row per artifact (draft, figure, table, script, note, reference), with creator flag (**human** / **agent** / **mixed**), model metadata for agent and mixed artifacts, and verification status. Every asset referenced by the final chapter is traceable here.

- `interaction-log.csv` — one row per non-trivial AI-assisted session from project inception through submission, with date, harness, model, input and output summaries, and assets affected. The log is the reconstructible record a reviewer needs to audit the AI's role.
- `background/literature/` — the materialized output of the lit-review protocol: `references.bib`, `classification.csv`, per-strand prompts and raw outputs under `ai-inputs/`, researcher-sourced material under `human-inputs/`, and per-source notes at `<citekey>.md`.
- `background/concepts/` and `background/feedback/` — concept notes and colleague, advisor, and reviewer feedback carried across sessions.
- `drafts/` — including `ai-agent-draft.tex`, an early agent-produced draft of this chapter, preserved alongside the final version so a reader can diff the two.
- `figures/`, `tables/`, `scripts/` — sources and derived artifacts for every figure and table the chapter prints.
- `appendix/` — the three skills above in Markdown plus the startup checklist, preserved in machine-readable form so a future researcher can drop them into their own skill library.

A reader who treats this chapter as a methodological reference can inspect the project folder itself as a worked example of the practices advocated: the same folder layout, the same asset registry, the same interaction log, scaled to a published paper rather than a toy.